

Context

This is Phase 4 (live, shadow/paper mode) of the validation roadmap. The FPGA-equivalent software strategy (`host/strategy_sw.py`, bit-identical to `strategy_imbalance.sv`) was driven from a *real* market-data feed and routed to a *real* paper broker. The pipeline ran end-to-end and placed live paper orders — but only after a series of integration and real-market-microstructure issues that simulation and dry-run could never surface. This document catalogues every one of them.

Feed: Alpaca free tier (real-time IEX equities + paper account), chosen over IBKR because no IB Gateway desktop app was installed on the VM. **Symbol:** AAPL. **Date of runs:** 17 June 2026, US market hours.

Issue Summary

#	Issue	Category	Status
1	No retail access to raw NASDAQ ITCH	Architectural	BY DESIGN
2	IB Gateway absent on VM	Environment	RESOLVED
3	Alpaca free tier: one concurrent connection	Feed limit	MITIGATED
4	IEX quotes flow only in market hours; lulls	Feed behavior	BY DESIGN
5	Buffered output & hung websocket shutdown	Tooling	RESOLVED
6	Wash-trade rejections (self-crossing orders)	Microstructure	RESOLVED
7	Insufficient quantity / position drift	Order mgmt	MITIGATED
8	Fat-finger guard halts after price drift	Risk model	MITIGATED
9	Signal skew / position pinned at limit	Strategy tune	MITIGATED
10	API keys shared in plaintext	Security	OPEN

Detailed Findings

#1 No retail access to raw NASDAQ TotalView-ITCH **BY DESIGN**
Symptom: The FPGA `itch_parser.sv` expects the binary NASDAQ ITCH feed, which is not available to a retail account (requires exchange membership / colocation).
Root cause: ITCH is an exchange-direct product; retail brokers expose only normalized top-of-book / depth APIs.
Resolution: Accepted as a project premise. The live path feeds the *software* strategy mirror, not `itch_parser.sv`. The FPGA's 205 ns tick-to-trade stays a *measured compute benchmark* shown alongside the live feed — never claimed to be in the retail critical path. Only paid Databento offers real ITCH binary if true hardware-parsing is ever wanted.

#2 IB Gateway not installed on the VM **RESOLVED**
Symptom: `live_feed.py -symbol AAPL` could not connect; nothing listening on port 4002/7497. Filesystem search found no `~/Jts` install, only the unrelated `ibus` input daemon.
Root cause: The IBKR path needs the IB Gateway *desktop* app running and logged into a paper account; it was never installed here (there is no web login that opens the API socket).
Resolution: Switched the live feed to Alpaca, which needs only an API key — no desktop gateway, no socket port. Added an `-alpaca` mode to `live_feed.py` reusing the identical `_process()` strategy/risk core.

#3 Alpaca free tier allows one concurrent data connection **MITIGATED**
Symptom: A second `StockDataStream` opened while the 5-minute soak was still streaming would be refused.
Root cause: Free-tier websocket connection limit (1).
Mitigation: Run feed sessions strictly sequentially; never overlap a threshold experiment, soak, or execution run with another live stream.

#4 IEX quotes flow only during market hours; quiet lulls

BY DESIGN

Symptom: A threshold experiment at $5.0\times$ recorded **0 ticks** in its window — not because of the threshold but because quotes briefly dried up.

Root cause: The free IEX feed only streams 09:30–16:00 ET, and even then a single symbol can go quiet for seconds at a time.

Resolution: Interpret 0-tick windows as data lulls, not strategy behavior. For off-hours plumbing tests use `-simulate` (synthetic continuous quotes).

#5 Buffered output and a hung websocket shutdown

RESOLVED

Symptom: The first live run appeared to “hang” and produced no log output; the process outlived its requested duration.

Root cause: (a) stdout was block-buffered through the pipe, so lines never flushed before the kill; (b) the websocket loop did not honor the duration cleanly on close.

Resolution: Run with `python -u` (unbuffered), bound the session with `asyncio.wait_for(stream._run_forever(), timeout=duration)`, and wrap external invocations in a shell `timeout -k` as a hard backstop. The session summary (`_report()`) now prints reliably at the end.

#6 Wash-trade rejections — the strategy crosses its own orders

RESOLVED

Symptom: 78 of 112 order attempts in the execution run were rejected with Alpaca error 40310000: “*potential wash trade detected... sell order exists, buy limit price should be less than existing sell limit price.*”

Root cause: The imbalance strategy fires BUY then SELL (and vice-versa) within milliseconds. Each order is an aggressive limit (BUY lifts the ask, SELL hits the bid), so a new order would trade against the account’s *own* resting limit order on the opposite side — a wash trade, which the broker blocks by market-integrity rule.

Fix (implemented): `_cancel_opposite()` cancels our still-working orders on the opposite side before each submit (cancel-on-new). In a re-run on the same BUY/SELL-mixed regime that previously produced 78 rejects, wash-trade rejections dropped to **0** (34 cancel-on-new actions fired). A strategy/execution-layer fix, independent of the FPGA datapath.

#7 Insufficient quantity available for a SELL

MITIGATED

Symptom: 40310000: “*insufficient qty available for order (requested: 100, available: 50)... held_for_orders: 240.*”

Root cause: The strategy issues SELLS sized by imbalance without reconciling against the actual held position and shares already committed to resting orders. The hardware-mirrored risk check enforces a *net position* limit but not per-order inventory availability at the broker.

Partial fix (implemented): an inventory clamp sizes each order to the net-position capacity left in its direction (clamp to 0 = skip), so the strategy trades up to the cap and stops instead of overshooting or having whole orders rejected. This cut insufficient-qty failures from many to **4** in the re-run.

Diagnosis: the first clamp used an *optimistic local* position from `record_fill()` (assumes full, instant fills). Real broker fills drift from it — a run ended at +1,195 shares while the local count read the 1,000 cap, and that local-vs-broker divergence caused the residual 4 failures.

Fix (implemented): `_sync_position()` reads the broker’s actual signed position for the symbol (`get_open_position`; 404 $\Rightarrow$ flat) and overwrites the risk guard’s count before every order, so the clamp sizes against reality. In the verification run this fired 7 position syncs and brought insufficient-qty failures to **0**. Trades one HTTP read of latency on the order path (irrelevant vs. the ms-scale network RTT) for correctness. Not an RTL bug.

Heavy-burst facets (found on re-test) + further fixes: a heavy *one-sided* burst (193 SELL / 10 BUY, 203 signals) exposed two more resting-order pathologies the calm runs never hit — (i) dozens of same-side limit orders rested and the broker locked their shares (`held_for_orders`), rejecting further orders; (ii) those stale resting orders filled *behind* new ones, overshooting the cap (−1,110 vs −1,000). Fixes: switch to **IOC** (immediate-or-cancel) orders so they never rest (no locked shares, deterministic per-order position change); and a **no-zero-cross clamp** (sell at most the long held / buy at most the short held in one order, extend in-direction only to the cap) which also keeps SELL size $\leq$ held inventory under IOC partial fills. The clamp is unit-verified (8/8 cases: never crosses zero, always respects cap). *Live re-verification of the combined IOC + no-cross stack is pending the next market session — the re-test window hit the 16:00 ET close (0 ticks).*

Residue: because the aggressive marketable limits filled immediately (no orders were left resting), the run ended flat-less — a **net −150-share AAPL short** (avg \$297.32). It was flattened afterwards with a single closing market order; account returned to 0 positions / 0 open orders.

#8 Fat-finger guard halts trading after price drift

MITIGATED

Symptom: During the 5-minute soak, 1,327 orders were blocked with `fat_finger`: price out of band once AAPL drifted from \$296.46 (the seed) to \$297+.

Root cause: The reference price is seeded once from the first valid mid and never updated — a faithful mirror of `risk_check.sv`, whose price band ($\pm \$0.50$, $\approx 0.33\%$) is measured against a *fixed* reference. Any sustained move past the band trips every subsequent order.

Mitigation & candidate enhancement: expected behavior for the current RTL; documented as such. A future RTL/SW enhancement would track the reference (e.g. slow EMA of mid) or express the band relative to the live mid, so the guard catches genuine fat-finger spikes without halting on normal drift.

#9 Signal skew and position pinned at the limit

MITIGATED

Symptom: On a lopsided book the strategy fired 137 BUY vs 10 SELL and hit the `MAX_POSITION = 1000`-share cap almost immediately; later soak ran 1,150 BUY vs 185 SELL.

Root cause: At the default $1.5\times$ threshold the trigger is easily met, so a persistently imbalanced book produces one-sided signals; the risk cap then correctly stops further accumulation.

Mitigation: `-threshold` tuning confirmed — raising it ($3.0\times$, $5.0\times$) yields fewer, more selective, more two-sided signals. Pick the threshold to match the desired trade rate for the demo.

#10 API keys shared in plaintext

OPEN

Symptom: Alpaca paper API key + secret were pasted into the chat session.

Root cause: Expedient during setup. Paper keys cannot move real money, but they grant full control of the paper account and data subscription.

Recommended fix: regenerate the key pair in the Alpaca dashboard; going forward pass credentials only via environment variables (`ALPACA_API_KEY` / `ALPACA_SECRET_KEY`), never in code or chat.

What Worked — Milestone Achieved

Despite the issues above, the end-to-end objective was met: **live IEX quotes → FPGA-equivalent strategy → hardware-mirrored pre-trade risk → real paper orders on Alpaca account PA3812EGI0G4.**

- 5-minute soak: 5,146 ticks, 1,335 signals, captured to `reports/soak_aapl.log`.
- Execution run: **34 paper orders placed** (real order IDs / status `PENDING_NEW`); 78 rejected for the microstructure reasons above.
- Measured compute latency: **FPGA 205 ns vs. software host $\approx 9\text{--}10\ \mu\text{s}$ mean** $\Rightarrow \approx 45\times$ faster on hardware (live order path then adds network RTT of tens of ms — the gap the project exists to illustrate).

Next Actions

1. **Done:** order-management layer — cancel-on-new (cleared #6, wash trades $\rightarrow 0$), broker-position sync, IOC orders, and a no-zero-cross inventory clamp (unit-verified) for #7.
2. **Pending:** live re-verify the IOC + no-cross stack next market session (re-test hit the 16:00 ET close). Let the queued SELL-74 auto-flatten at next open.
3. **Done:** flattened the -150 -share AAPL short left by the execution run (the orders filled immediately rather than resting); account is back to 0 positions / 0 open orders.
4. Rotate the Alpaca API keys (#10).
5. Feed `reports/soak_aapl.log` into `notebooks/latency_histogram.ipynb` for the live-data dashboard.
6. (Stretch) reference-tracking price band (#8) in RTL + software mirror.